Introduction to Programming, Variables, and Operators

1. About Java Programming Language

Java is a popular, object-oriented programming language that was developed by Sun Microsystems (now owned by Oracle) in 1995. Here are key characteristics of Java:

Key Features:

- **Platform Independent**: Java programs can run on any device that has the Java Virtual Machine (JVM) installed
- **Object-Oriented**: Everything in Java is organized around objects and classes
- **Simple and Secure**: Java has a clean syntax and built-in security features
- **Robust**: Java has strong memory management and error handling

How Java Works:

1. You write Java source code in `.java` files
2. The Java compiler (`javac`) converts your code into bytecode (`.class` files)
3. The Java Virtual Machine (JVM) executes the bytecode

Basic Structure of a Java Program:

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

2. IDE (Integrated Development Environment)

An IDE is a software application that provides comprehensive tools for software development. It combines several development tools into a single interface.

What an IDE includes:

- **Text Editor**: For writing code with syntax highlighting
- **Compiler**: To convert source code into executable code
- **Debugger**: To find and fix errors in your code

- **Project Management**: To organize your files and folders

Popular Java IDEs:

- **Eclipse**: Free and widely used
- **IntelliJ IDEA**: Powerful with smart code completion
- **NetBeans**: User-friendly for beginners
- **Visual Studio Code**: Lightweight with Java extensions

Benefits of using an IDE:

- Syntax highlighting makes code easier to read
- Auto-completion helps you write code faster
- Error detection shows mistakes as you type
- Easy project organization and file management

3. System.out.println() vs System.out.print()

Both methods are used to display output to the console, but they behave differently.

System.out.println()

- Prints text and adds a **new line** at the end
- The cursor moves to the next line after printing

```java
System.out.println("First line");
System.out.println("Second line");
```

Output:

```
First line
Second line
```

System.out.print()

- Prints text **without** adding a new line
- The cursor stays on the same line

```java
System.out.print("Hello ");
System.out.print("World");
```

Output:

```
Hello World
```

Comparison Example:

```java
public class OutputExample {
    public static void main(String[] args) {
        System.out.print("This is ");
        System.out.print("on the same line. ");
        System.out.println("This ends the line.");
        System.out.println("This is on a new line.");
    }
}
```

Output:

```
This is on the same line. This ends the line.
This is on a new line.
```

4. Variables

A variable is a container that holds data that can be changed during program execution. Think of it as a labeled box where you can store different values.

Variable Declaration and Initialization

Declaration: Creating a variable

```java
int age;  // Declares a variable named 'age' of type int
```

Initialization: Assigning a value to the variable

```java
age = 25;  // Assigns the value 25 to the variable 'age'
```

Declaration and Initialization Together:

```java
int age = 25;  // Declares and initializes in one line
```

Common Data Types in Java

Primitive Data Types:

- `int`: Whole numbers (e.g., 42, -17, 0)
- `double`: Decimal numbers (e.g., 3.14, -2.5)
- `char`: Single characters (e.g., 'A', '5', '$')
- `boolean`: True or false values
- `String`: Text (e.g., "Hello World")

Variable Examples:

```java
public class VariableExample {
    public static void main(String[] args) {
        // Integer variable
        int studentCount = 30;

        // Double variable
        double price = 19.99;

        // Character variable
        char grade = 'A';

        // Boolean variable
        boolean isActive = true;

        // String variable
        String name = "John";

        // Display the variables
        System.out.println("Student Count: " + studentCount);
        System.out.println("Price: $" + price);
        System.out.println("Grade: " + grade);
        System.out.println("Is Active: " + isActive);
        System.out.println("Name: " + name);
    }
}
```

Variable Naming Rules:

- Must start with a letter, underscore (_), or dollar sign ($)
- Cannot start with a number
- Cannot use Java keywords (like `int`, `class`, `public`)
- Case-sensitive (`age` and `Age` are different variables)
- Use meaningful names (`studentAge` instead of `x`)

5. Operators (+, -, *, /, =, %)

Operators are symbols that perform operations on variables and values.

Arithmetic Operators

Addition (+): Adds two values

```java
int sum = 10 + 5;  // sum = 15
```

Subtraction (-): Subtracts the second value from the first

```java
int difference = 10 - 3;  // difference = 7
```

Multiplication (*): Multiplies two values

```java
int product = 6 * 4;  // product = 24
```

Division (/): Divides the first value by the second

```java
int quotient = 20 / 4;  // quotient = 5
double result = 7.0 / 2.0;  // result = 3.5
```

Modulus (%): Returns the remainder after division

```java
int remainder = 17 % 5;  // remainder = 2
```

Assignment Operator

Assignment (=): Assigns a value to a variable

```java
int x = 10;  // Assigns 10 to variable x
x = 20;      // Changes the value of x to 20
```

Operator Examples:

```java
public class OperatorExample {
    public static void main(String[] args) {
        int a = 15;
        int b = 4;

        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("a + b = " + (a + b));  // 19
        System.out.println("a - b = " + (a - b));  // 11
        System.out.println("a * b = " + (a * b));  // 60
        System.out.println("a / b = " + (a / b));  // 3
        System.out.println("a % b = " + (a % b));  // 3
    }
}
```

6. Difference between / and %

Understanding the difference between division (/) and modulus (%) operators is crucial for programming.

Division Operator (/)

- Returns the **quotient** (the result of division)
- When dividing integers, it returns only the whole number part
- When dividing doubles, it returns the exact decimal result

```java
// Integer division
int result1 = 17 / 5;    // result1 = 3 (whole number only)
int result2 = 20 / 4;    // result2 = 5

// Double division
double result3 = 17.0 / 5.0;  // result3 = 3.4
double result4 = 20.0 / 4.0;  // result4 = 5.0
```

Modulus Operator (%)

- Returns the **remainder** after division
- Useful for finding if a number is even or odd
- Helpful for creating repeating patterns

```java
int remainder1 = 17 % 5;  // remainder1 = 2
int remainder2 = 20 % 4;  // remainder2 = 0
int remainder3 = 10 % 3;  // remainder3 = 1
```

Visual Example:

When you divide 17 by 5:

- 17 ÷ 5 = 3 remainder 2
- `/` gives you the 3 (quotient)
- `%` gives you the 2 (remainder)

Practical Applications:

```java
public class DivisionModulusExample {
    public static void main(String[] args) {
        int number = 23;

        // Check if number is even or odd
        if (number % 2 == 0) {
            System.out.println(number + " is even");
        } else {
            System.out.println(number + " is odd");
        }

        // Convert minutes to hours and minutes
        int totalMinutes = 150;
        int hours = totalMinutes / 60;      // 2 hours
        int minutes = totalMinutes % 60;    // 30 minutes

        System.out.println(totalMinutes + " minutes = " + hours + " hours and " + minutes + " m
    }
}
```

Quiz Questions

1. Which of the following is NOT a characteristic of Java? a) Platform independent
b) Object-oriented

c) Requires manual memory management
d) Secure

Answer: c) Requires manual memory management

2. What is the difference between System.out.print() and System.out.println()? a) There is no difference
b) System.out.print() adds a new line, System.out.println() doesn't
c) System.out.println() adds a new line, System.out.print() doesn't
d) System.out.print() is faster than System.out.println()

Answer: c) System.out.println() adds a new line, System.out.print() doesn't

3. What will be the value of `result` after executing: `int result = 25 % 7;`? a) 3
b) 4
c) 7
d) 25

Answer: b) 4

4. Which of the following is a valid variable name in Java? a) 2student
b) student-name
c) student_name
d) class

Answer: c) student_name

5. What will be the output of: `System.out.println(15 / 4);` (assuming integer division)? a) 3.75
b) 3
c) 4
d) 15

Answer: b) 3

Practice Coding Questions

1. Write a Java program that declares variables for your name (String), age (int), and height in meters (double). Display all three variables using System.out.println().

2. Create a program that calculates and displays the area of a rectangle. Declare variables for length and width, assign them values, and use the formula: area = length × width.

3. Write a program that takes a number of seconds and converts it to hours, minutes, and remaining seconds. For example, 3665 seconds should display as "1 hours, 1 minutes, and 5 seconds."

4. Create a program that determines if a given number is even or odd using the modulus operator. Display an appropriate message.

5. Write a program that calculates the average of three test scores. Declare three variables for the scores, calculate the average, and display the result with an appropriate message.